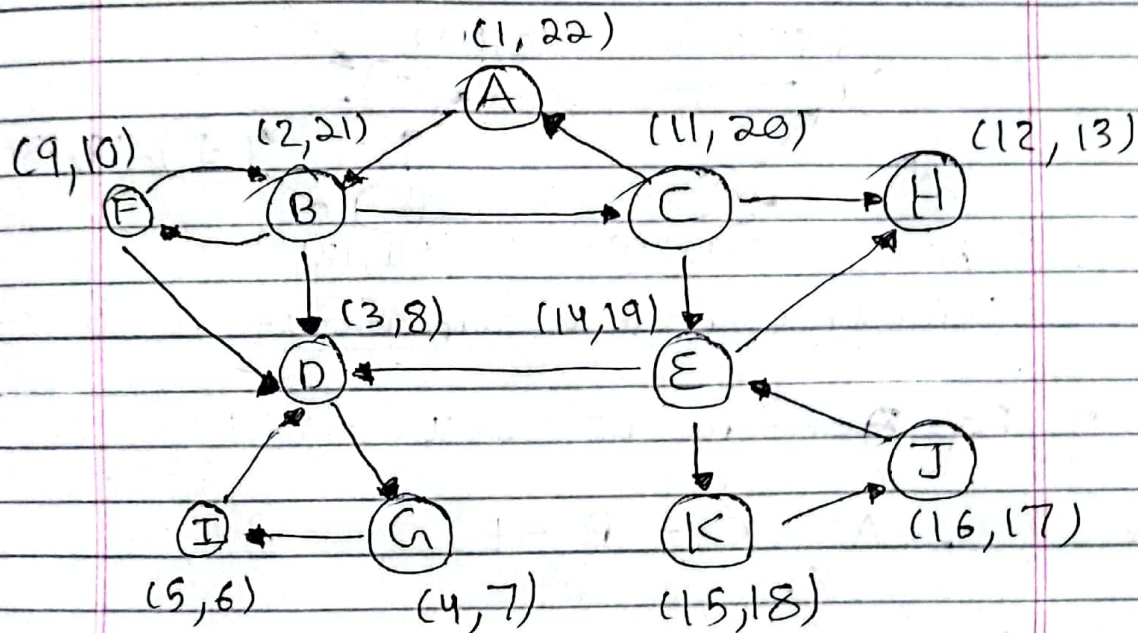


Homework 6

Design and Analysis of Algorithms

Muhammad Hassan Khalid
211-5692

Question 1



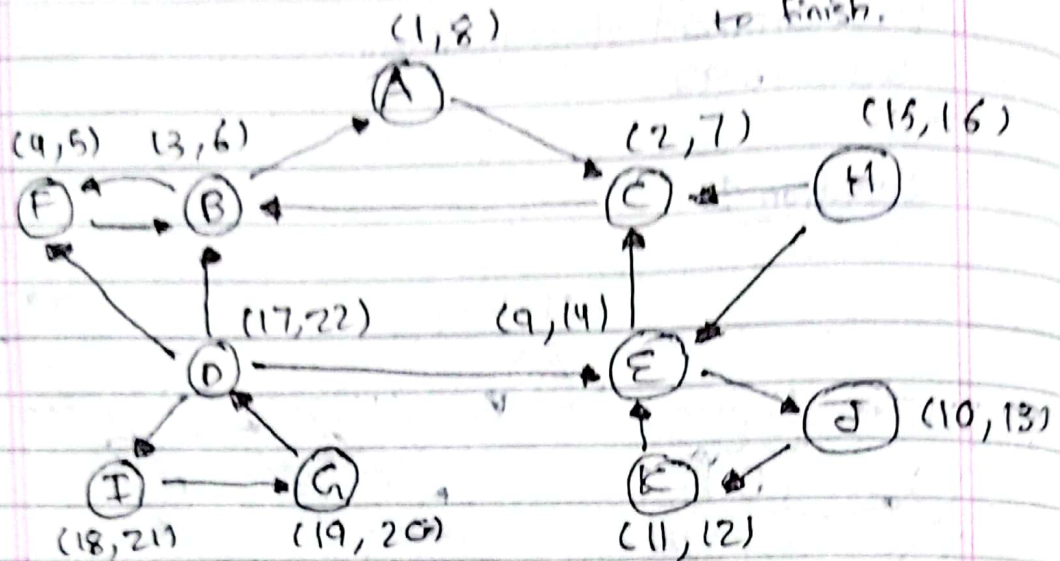
Stack

A
B
C
E
K
J
H
F
D
G
I

The first iteration of DFS
took 22 seconds.

REVERSED GRAPH

The 2nd iteration
also took
27 seconds
to finish.



Strongly Connected Components :-

→ Pop ~~A~~ and run DFS on it

1. A - C - B - F

→ Pop B and C but don't run DFS as already visited

→ Pop E and run DFS on it.

2. E - J - K

→ Pop K and J but don't run DFS as already visited

→ Pop H and run DFS on it

3. H

→ Pop F but don't run DFS as already visited

→ Pop D and run DFS on it.

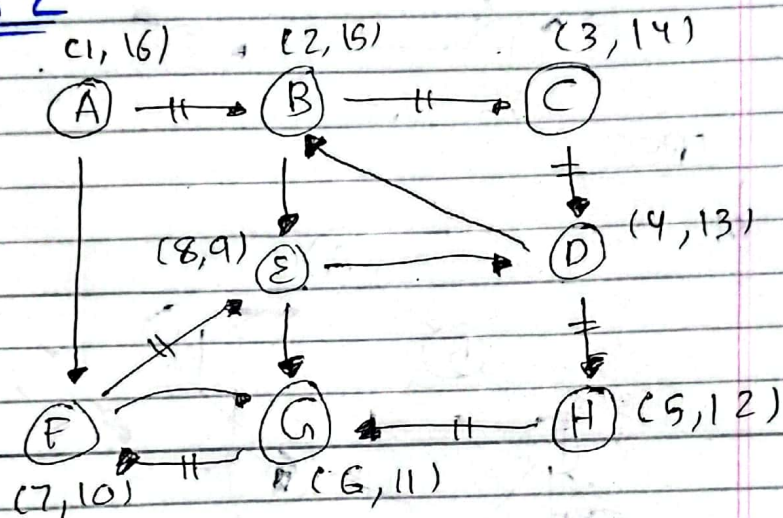
4. D - I - G

SCC's

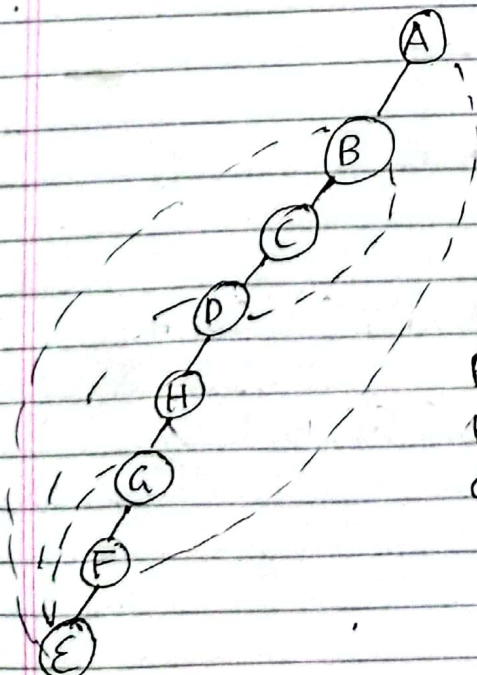
- 1 ACBF
- 2 EKJ
- 3 H
- 4 DIG

Question 2

A



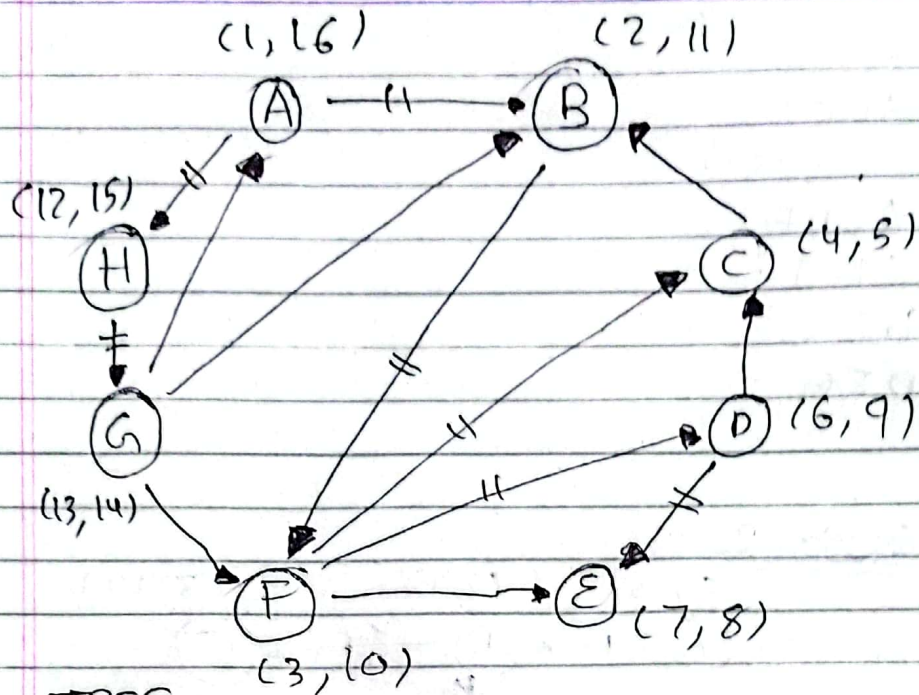
TREE



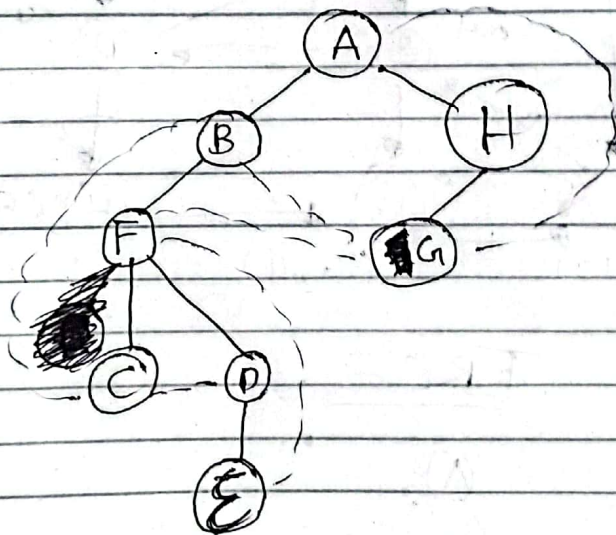
Edges:-

Backward: (E, D), (D, B), (F, G)
 Forward: (B, E), (A, F)
 Cross: NONE

B



TREE



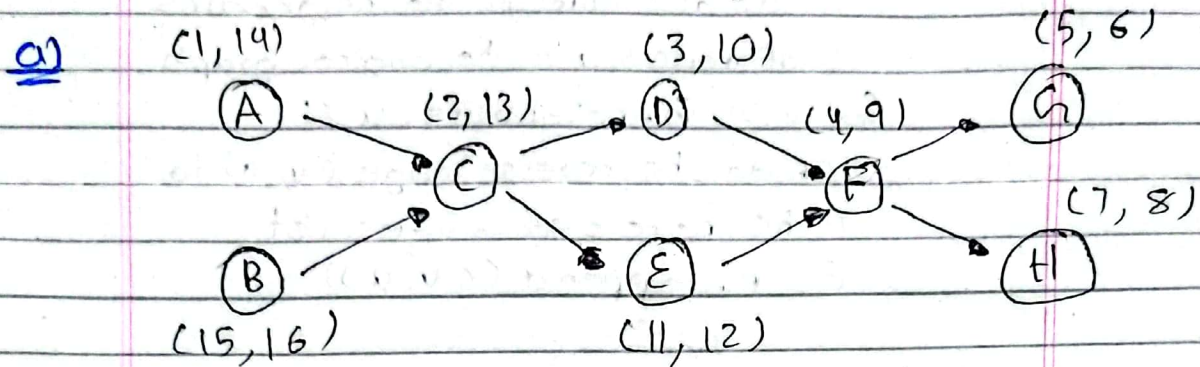
Edges

Backward: $(C, B), (D, C), (G, A)$

Forward: (F, E)

Cross: $(G, F), (G, B)$

Question 3



b) Sources = A and B
Sink S = G and H

c) B A C E D F H G

d) 3 interchangeable vertices
 $\{A, B\}$, $\{D, E\}$, $\{G, H\}$
 leading to, $2^3 = 8$ possible combinations.

Stack

B
A
G
E
D
F
H
G

Question 4

function reverseGraph(G);

// G is the original graph in adjacency

// list format

V = G.vertices

E_R = empty list // Initialize the

// reversed edges list

// Iterate through each vertex in the
 // original graph

for u in V :

// Iterate through the neighbours

// of vertex u in the original graph

for v in G .neighbours(u):

// Add the reverse edge (v, u) to

// the reversed edges list

E_R .append((v, u))

// Create the reverse graph G_R using

// the reversed edges list

$G_R = \text{Graph}(V, E_R)$

return G_R

Question 5

9

For all the vertices we are first assigning them infinity and the source node as 0, so the time complexity is $O(V)$.

Secondly, we are building a min heap hence again $O(V)$ time complexity is there.

To extract all vertices from a heap time complexity would be $O(V \log V)$.

Finally to relax all vertices the time complexity would be $O(E \cdot \log V)$.

$$\begin{aligned} \text{Time complexity} &= O(V) + O(V) + V \log V \\ &\quad + E \cdot \log V \\ &= O(E \cdot \log V). \end{aligned}$$

So the statement is false, Dijkstra's shortest path algorithm does not take $O(|V| \lg |V|)$ time on a lexicograph.

Dijkstra's (Graph, Source)

Create vertex set Q

for each vertex v in graph

$\text{dist}[v] = \infty$

add v to Q

$\text{dist}[\text{source}] = 0$

$O(n)$

$O(V)$

Build heap

while Q is not empty

$u = \text{Extract_min}(Q)$

$O(\log V)$

for each neighbour v of u

$\text{Relax}(u, v)$

$O(E \cdot \log V)$

In the Korchoff graph, the number of edges can be much larger than the number of vertices due to the presence of edges going from each leaf to the root of the binary tree.

b

Since the Korchoff graph has a binary tree at its center, we can perform a depth-first traversal of the binary tree to compute the shortest path. The idea is to start from the root of the binary tree and propagate the shortest path information down to the leaves. Then, for each leaf, update the shortest paths from the leaf to the root.

Algorithm in English:

1. Perform a depth-first traversal of the binary tree starting from the root.
2. During the traversal, compute the shortest paths from the root to each node.

3. For each leaf, update the shortest paths from the leaf to the root.
4. Return the computed shortest paths.

Pseudocode:

```
function spkarchoff (G, s, r):
    for each vertex v in G.V:
        v.dist = infinity
        v.parent = null
```

```
// perform DFS starting from the root
DFS (G, r, s, G)
```

```
function DFS (G, current, s, accumulatedWeight):
    current.dist = accumulatedWeight
    for each neighbour in G.adjacencyList(current):
        DFS(G, neighbour, s, accumulatedWeight + weight(current, neighbor))
```

```
// update the shortest paths from
// leaves to the root
if isLeaf(current):
    updateShortestPaths(G, s, current)
```

```
function updateShortestPaths (G, s, leaf):
    current = leaf
    while current.parent is not null:
        if current.dist < G.V[current.parent].dist:
            G.V[current.parent].dist = current.dist
        current = G.V[current.parent]
```

This algorithm works in linear time because each vertex is visited and processed exactly once during DFS traversal, resulting in a time complexity $O(V)$.

Dijkstra's algorithm can be used, the basic idea is to find the shortest path from Bob's house A to each hotel, considering the risk values as the edge weights and then use these shortest paths to determine the path for all agents.

Question 6

Algorithm

1. Initialize an empty priority queue Q and a dictionary $dist$ to store the shortest distance from A to each vertex. Set $dist[A] = 0$ and $dist[v] = \infty$ for all other vertices v .
2. Add A to Q with priority 0.
3. While Q is not empty:
 4. Extract the vertex u with the minimum priority from Q .
 5. For each neighbour v of u :
 - a. Calculate the new distance to v as $dist[v] = dist[u] + weight(u, v)$ / where $weight(u, v)$ is the risk value associated with the edge (u, v) .
 - b. If the new distance is smaller than the current distance $dist[v]$, update $dist[v]$ with the new distance and add v to Q with priority $dist[v]$.
6. After the Dijkstra's algorithm terminates, $dist$ will contain the shortest distances from A to all vertices.

Now, to determine the path for all agents, use the shortest distances computed by Dijkstra's algorithm. For each hotel h_i , trace back the shortest path from A to h_i using the dist dictionary. This will give us the path with the minimum risk for each agent.

Pseudocode:

1 For each hotel h_i :

- a Start with an empty path for agent i .
- b Initialize the current vertex v as h_i .
- c While v is not equal to A
 - i Add v to the beginning of the path for agent i .
 - ii Find the neighbour u of v that minimizes $\text{dist}[u] + \text{weight}(u, v)$.
 - iii Set v as u .
- d. Add A to the beginning of the path for agent i .

The path for all agents are now determined.